paper.docx

Dr. D. Y. Patil Institute of Technology, Pipmri

Document Details

Submission ID

trn:oid:::3618:126184207

Submission Date

Jan 14, 2026, 4:29 PM GMT+5:30

Download Date

Jan 14, 2026, 4:30 PM GMT+5:30

File Name

paper.docx

File Size

232.6 KB

6 Pages

3,960 Words

22,750 Characters

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **16** Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- **2** Missing Quotations 0%
  Matches that are still very similar to source material
- **4** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
9% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **16** Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- **2** Missing Quotations 0%
  Matches that are still very similar to source material
- **4** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
9% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Publication	Priya Pujari, Roshani Ade. "Enhancing performance of keyword query over struct...	1%
2	Internet	beei.org	1%
3	Internet	www.researchsquare.com	<1%
4	Internet	www.ijfmr.com	<1%
5	Internet	test.techscience.com	<1%
6	Internet	www.techscience.com	<1%
7	Internet	self-issued.info	<1%
8	Internet	www.sersc.org	<1%
9	Publication	"Blockchain and Trustworthy Systems", Springer Science and Business Media LLC,...	<1%
10	Internet	www.jetir.org	<1%

11 Internet

e-archivo.uc3m.es <1%

12 Internet

iarjset.com <1%

13 Internet

www.ijert.org <1%

14 Internet

export.arxiv.org <1%

15 Internet

www.researchgate.net <1%

16 Publication

"Advances in Artificial Intelligence and Security", Springer Science and Business ... <1%

17 Publication

Haughton, Odayne R.. "The Novel Proof of Efficiency (PoEf) Consensus Mechanism... <1%

18 Publication

Mukta, Rahma Bintey Mufiz. "Privacy Preserving Identity and Credential Manage... <1%

19 Internet

thesai.org <1%

20 Publication

V. Sharmila, S. Kannadhasan, A. Rajiv Kannan, P. Sivakumar, V. Vennila. "Challeng... <1%

Adaptive Encryption and Dual-Hash Blockchain Framework for Secure File Sharing

Sharad Adsure
Department of Computer Engineering
Dr. D.Y. Patil Institute of Technology
Pune, Maharashtra, India
sharad.adsure@dypvp.edu.in

Ansh Jaiswal
Department of Computer Engineering
Dr. D.Y. Patil Institute of Technology
Pune, Maharashtra, India
anshj0044@gmail.com

Nihar Patil
Department of Computer Engineering
Dr. D.Y. Patil Institute of Technology
Pune, Maharashtra, India
niharpatil2005@gmail.com

Madhura Patil
Department of Computer Engineering
Dr. D.Y. Patil Institute of Technology
Pune, Maharashtra, India
patilmadhura2701@gmail.com

Gunjan Patil
Department of Computer Engineering
Dr. D.Y. Patil Institute of Technology
Pune, Maharashtra, India
gunjanpatil274@gmail.com

***Abstract*—The need for secure file sharing has become a great necessity in today's organizations as the number of data leaks, unauthorized access, and tampering incidents in traditional cloud-based systems goes up. The use of centralized storage systems leads to single points of failure and restricted transparency in access auditing. This paper presents a secure file sharing framework that is decentralized, using Adaptive Encryption and Dual-Hash Verification in conjunction with Blockchain and IPFS to solve these problems. The suggested method is such that the file is first encrypted at the sender's end via an adaptive AES mechanism that selects either AES-128, AES-192, or AES-256 according to the sensitivity and size of the file to have a good security–performance trade-off. The file encrypted is kept in IPFS, that is off chain, while the blockchain holds just necessary metadata such as the CID, timestamp, access policy, and integrity proofs. A dual-hash process is used to check the integrity of both encrypted file content and metadata this way preventing any silent data alteration and manipulating of metadata. Besides, a rule-based anomaly detection module keeps watch over access behaviour and sets off a warning for suspicious download attempts, which are for instance, repeated access, device mismatch, or unusual activity pattern. The proposed framework guarantees confidentiality, integrity, availability, and auditability; thus, it can be used for the secure sharing of sensitive files in the academic, enterprise, and government sectors.**

Keywords*—Secure File Sharing, Blockchain, IPFS, Adaptive Encryption, Dual-Hash Verification, Anomaly Detection

I. Introduction

File sharing that is secure and reliable has become a necessity among organizations, educational institutions, healthcare and governmental sectors that have all led to the digital communication and remote collaboration to be at such a high level that it cannot be reversed. The customary systems for file sharing are mostly based on the cloud and they are under central control, because the files are stored on the servers of a third party. Despite being user-friendly, these systems have to deal with some severe security problems like data leakage, insider threats, unauthorized modification and lack of transparency in access auditing [1]. In addition, the centralized infrastructure results in a single point of failure which makes it susceptible to service disruption and denial-of-service attacks [2]. Blockchain and distributed storage have been able to solve these challenges, by using decentralized architectures, which are inheriting the advantages of both systems.

The combination of blockchain and decentralized storage solves all the drawbacks of the current solutions. A blockchain creates a secure and unchanged ledger, which allows recording metadata, integrity proofs, and access logs in an auditable manner, while smart contracts can automatically enforce access and keep the records verifiable [3], [4]. To solve the problem of impossibility of storing full files on-chain, the system uses IPFS for off-chain storage and blockchain stores essential identifiers such as CIDs and integrity hashes [5]. The IPFS content addressing not only ensures the detection of tampering but also provides a history of changes made through CID alterations [6]. On the other hand, confidentiality and cryptographic efficiency have not been resolved yet, because the use of static encryption (e.g., AES-256 for all files) results in an unavoidable overhead [7]. Moreover, the manipulation of metadata and the occurrence of suspicious access will be monitored through behaviour-aware systems [8].

The current work suggests a combination of adaptive encryption and dual-hash blockchain along with rule-based anomaly detection to improve integrity, confidentiality, and enforcement [7], [9], [10]. This research has made the following primary contributions:

- New adaptive encryption system choosing AES strength on-the-fly depending on file size and sensitivity in order to cut down on unnecessary overhead.
- Simultaneous dual-hash integrity verification making it possible to protect encrypted file content and metadata at the same time.
- Combination of blockchain and IPFS architecture where only a lightweight part of metadata is kept on-chain, and the rest of the encrypted data is placed off-chain to enhance scalability.
- Anomaly detection based on rules enabling the very low cost of computation to give real-time detection of the suspicious access patterns.

II. Related Work

The study of decentralized file storage and secure file sharing has gained a lot of attention as a potential solution for the security and reliability issues of traditional cloud systems. Blockchain technology gives the possibility of making file metadata and access events recording immutable, transparent and auditable while the work of Satoshi Nakamoto opened the way for the idea of a tamper-proof distributed ledger [3]. IPFS has attracted a lot of interest as a decentralized storage layer

that could work in parallel with; Benet described IPFS as a content-addressed peer-to-peer storage network where files are denoted by a Content Identifier (CID), and any modification of the content leads to a change of the CID, thus enabling detection of the tampering [5][6]. Huang et al. put forward a blockchain–IPFS secure file sharing method that includes decentralized access control and group key management via an IPFS proxy [11]. In addition to this, similar approaches utilize blockchain smart contracts to regulate access rights, keep file metadata like CID/hash values, and back up tracking of user activities [12][13]. Notwithstanding, a number of current solutions still depend on static encryption policies, which not only incur unnecessary computational overhead but also impair performance when resources are limited or in times of high-volume [12].

On top of that, the majority of integrity mechanisms are mainly concerned with verifying the content of files and thus are not very effective against the alteration of metadata. The need for behaviour-aware enforcement is increasingly recognized in the new generation of cyber defense systems; Isolation Forest allows for unsupervised anomaly detection without the need for labelled data sets [8], whereas rule-based anomaly detection continues to be light, comprehensible and applicable to real-time enforcement in security applications [10].

The above-mentioned limitations are the driving force behind the suggested framework that combines adaptive encryption, dual-hash integrity verification, and behavioural anomaly monitoring.

III. Architecture

The suggested system design combines adaptive cryptography, decentralized storage and blockchain-based audit logging allowing secure file sharing without dependency on centralized servers. The structure is constructed in a modular fashion consisting of four essential modules: (1) File Input and Encryption, (2) Secure Storage and Ledger Registration, (3) Access Verification, and (4) Decryption and Download. Every module accomplishes a particular security task, and the total process offers confidentiality, integrity, availability, and accountability.

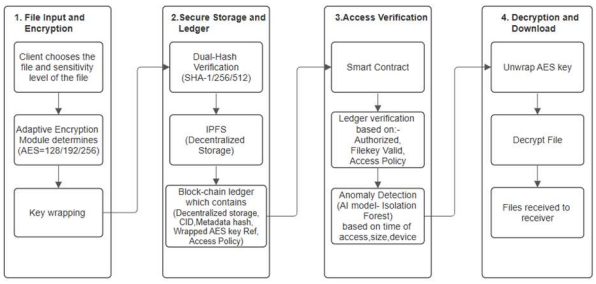

Fig. 1. System Architecture.

1. *Architecture Overview*

The workflow in its entirety starts from sender-side encryption and concludes with receiver-side verification and decryption. The system employes the following methods:

- Adaptive AES encryption to encrypt file content according to its confidentiality level and size,
- RSA key wrapping to securely transfer the AES key to the authorized receiver,
- IPFS to hold encrypted file content off chain using CID,
- A Blockchain smart contract for the storage of only the necessary metadata (CID + hashes + timestamp + access policy),
- Implementation of dual-hash verification for the detection of both file tampering and metadata alteration,
- And the setup of rule-based anomaly detection for the identification and blocking of suspicious access behaviour.

This hybrid architecture guarantees scalability (because of the off-chain storage) and trust (due to the blockchain immutability), alongside the efficient cryptographic enforcement.

2. *Module 1: Input and Encryption of Files*

The initial phase consists of the sender uploading a document and picking a document sensitivity level. The Adaptive Encryption Module (AEM) looks at file characteristics like its size and sensitivity and turns to an AES training path with the fastest configuration possible:

- AES-128 for data with low sensitivity (privacy at lightning speed, and no computation at all),
- AES-192 for data with medium sensitivity (security and speed are perfectly matched),
- AES-256 for very sensitive data or very big files (the strongest security).

An AES key and a random Initialization Vector (IV) are newly created for each file to provide high entropy and to avoid the same ciphertext. The AES key is then wrapped using the receiver's RSA public key after AES encryption which guarantees that only the intended receiver can use their private key to open the AES key. This hybrid cryptographic scheme marries the speed of the symmetric algorithm AES with the security of the RSA key exchange.

3. *Module 2: Secure Storage and Ledger Registration*

After encryption, both the encrypted file and its corresponding metadata undergo processing for decentralized storage and logging which cannot be changed.

a. Dual-Hash Verification

The following procedure is done:

- H_1 = SHA256(encrypted file); guarantees the integrity of encrypted content
- H_2 = SHA256(metadata JSON); guarantees the integrity of metadata

Through this dual-hash technique, the following attacks are prevented: ciphertext modification attacks, metadata manipulation (changing the receiver's identity / altering the timestamp / changing the policy). For any 1-bit alteration made in the content of the file or metadata, a different hash value will be produced thus making it possible to detect tampering effectively.

b. IPFS Storage

The encrypted file is uploaded to the IPFS which then assigns a unique Content Identifier (CID)

corresponding to the content of the file. If the file stored is altered, the CID will also be changed, and the storage system will thus have tamper-evident characteristics by default. Additionally, the use of the IPFS network ensures that data is always accessible because it can be downloaded from several different peers.

c. Blockchain Registration

The smart contract on the blockchain keeps track of only small metadata like:

- CID,
- H_1 and H_2,
- Date and Time,
- Identity of sender and receiver,
- Access policy.

This reduces the overhead of storing on the blockchain while maintaining the properties of auditing and verification records that are immutable and publicly verifiable.

4. Module 3: Access Verification

The system uses both ledger-based authorization and behavioral verification to validate the request when the recipient asks for permission to access a file.
The blockchain ledger is checked to verify:

- Identity of the receiver,
- Policy of access,
- Mapping of the file CID,
- Hashes that are stored.

The rule-based anomaly detection engine keeps track of access patterns and includes:

- Access time, which is not typical,
- Access frequency that is higher than usual,
- Device that does not match,
- Anomaly in location.

If suspicious behaviour is detected, access is denied and alarms are triggered. This module is just a security enhancer which does not need machine learning datasets or training overhead.

5. Module 4: Decryption and Download

Once the access verification process is completed successfully, the receiver gets hold of the encrypted document from IPFS through CID. The AES key is then retrieved by the RSA-wrapped key's decryption with the receiver's private key. The system decrypts the file with AES and checks its integrity by:

- Hashing the file again and checking it against the value on the blockchain.
- Verifying the hash consistency of the metadata.

The file is only released to the receiver after the integrity verification has been done successfully. Any inconsistency will notify tampering and stop the delivery of the file.

6. Component Interactions and Data Flow

The entire data flow is presented in a concise version below:

- User uploads a file: adaptive AES encryption + RSA key wrapping.
- Ledger registration: IPFS file upload; CID + hashes on blockchain.
- Access request: the sender's request for the file; smart contract + anomaly detector confirms the transfer.
- Retrieval and verification: the file is retrieved from IPFS; the AES key is unwrapped; the file is decrypted and verified.

This modular interaction enhances all aspects of file sharing by making them more traceable, extensible and securely enforceable.

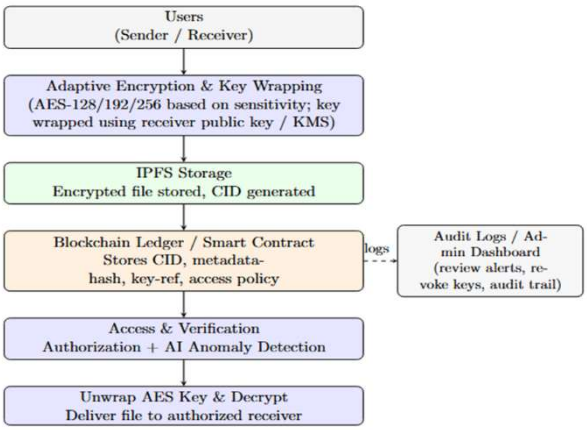

Fig. 2. System Workflow

IV. Algorithms

The algorithms that are part of the secure file-sharing framework proposed are disclosed in this section. The process is partitioned into activities performed by the sender (encryption, hashing, storage, and ledger registration) and those carried out by the receiver (verification, anomaly checks, and decryption).

1. Adaptive AES Encryption Selection

This algorithm provides high encryption for the user with demanding, large files and fast encryption for relatively smaller files.

- Input: File size $\boldsymbol{S}$, sensitivity level $\boldsymbol{L}$ (Low/Medium/High)
- Output: AES mode AES_k where $k \in \{128,192,256\}$
  1. If $L = High$ **then** select AES_{256}
  2. **Else if** $L = Medium$ **then** select AES_{192}
  3. **Else** select AES_{128}
  4. If $S > S_{threshold}$ and $L \neq Low$, upgrade to AES_{265}
  5. Return selected AES configuration.

2. Secure File Upload (Sender Side)

Data is strictly stored using encryption in IPFS protocol and file metadata is permanently saved on the blockchain.

- Input: File $\boldsymbol{F}$, receiver public key $\boldsymbol{PK_R}$, sensitivity level $\boldsymbol{L}$
- Output: Blockchain record ***TxID***, IPFS CID, encrypted AES key ***EncKey***
  1. Read input file $\boldsymbol{F}$ and determine file size $\boldsymbol{S}$.
  2. Choose AES mode using **Algorithm 1.**

3. Generate a random AES session key **K** and a random initialization vector **IV** for the selected AES mode.
4. Encrypt the file using AES:
$$C = AES_ENC(F, K, IV)$$
5. Encrypt the AES key using receiver's RSA public key:
$$EncKey = RSA_ENC(K, PK_R)$$
6. Construct metadata $M = \{SenderID, ReceiverID, Timestamp, AccessPolicy, AESMode\}$.
7. Compute dual hashes:
   - *H1=SHA256(C)*
   - *H2=SHA256(M)*
8. Upload ciphertext *C* to IPFS and obtain CID.
9. Store $\{CID, H_1, H_2, Timestamp, ReceiverID, Policy\}$ on blockchain via smart contract.
10. Return *TxID*, CID, EncKey.

3. Access Verification and Rule-Based Anomaly Detection

- Input: Access request
  $R = UserID, CID, DeviceID, Location, Time\}$
- Output: Access decision (Allow / Deny)
  1. Retrieve ledger entry corresponding to CID from blockchain.
  2. Verify that *UserID* matches authorized receiver.
  3. Initialize anomaly score *A = 0*
  4. Apply rule checks:
     - If access time is unusual, increment *A*.
     - If repeated access exceeds threshold, increment *A*.
     - If device mismatch is detected, increment *A*.
     - If location anomaly is detected, increment *A*.
  5. If $A \geq A_{threshold}$, deny access and log alert.
  6. Else allow access.

4. Secure File Download (Receiver Side)

Only the person who is authorized to get the file can decrypt it. Any attempts to tamper with the file will be noticed because of the dual-hash verification that is used.

- Input: CID, encrypted AES key EncKey, receiver private key SK_R
- Output: Decrypted file F′
  1. Request access and validate using **Algorithm 3.**
  2. Retrieve encrypted file *C* from IPFS using CID.
  3. Fetch stored hashes H_1, H_2 and metadata *M* from blockchain.
  4. Recompute integrity hashes:
     - $H_1' = SHA256(C)$
     - $H_2' = SHA256(M)$
  5. If $H_1' \neq H_1$ or $H_2' \neq H_2$, abort (tampering detected).
  6. Decrypt AES key using receiver's private key:
  $$K = RSA_DEC(EncKey, SK_R)$$
  7. Decrypt file using AES:
  $$F' = AES_DEC(C, K, IV)$$
  8. Return original file *F′*

V. Result and Discussion

Here are the experimental validations and observed results of the proposed secure file sharing architecture with Adaptive AES encryption, RSA key wrapping, dual- hash integrity validation, blockchain audit logging, IPFS storage and rule-based anomaly detection. A local deployment setup was used in the testing of the prototype where the blockchain network and IPFS node were deployed locally to test real-time secure storage, authorized sharing and tamper/anomaly detection.

A. Testing Prototype Environment

The structure was tested in a controlled setting as the following setup:

- Local blockchain network (private test chain).
- Local encrypted storage decentralized (pinning enabled) IPFS node.
- Application modules that are implemented using local server configuration (sender receiver scenario)
- Smart contract containing metadata and integrity proofs of a file.

This will make sure that the output is based on the practical delivery feasibility without relying on the external cloud services.

B. Secure File Sharing Validation

To test the service of safe sharing of files, the sender loaded several files and sent them to the receiver using the suggested workflow:

- Sender uploads file and finds level of sensitivity.
- Encryption of file is through adaptive AES.
- Encrypted file is stored into IPFS and represented by CID.
- Metadata + CID + hashes are registered in blockchain.
- Request Receivers get access to and download encrypted file in IPFS.
- Receives and verifies integrity and decrypts file with recovered AES key.

Observed result:

In IPFS, files were never stored without being encrypted. The shared file could only be accessed and decrypted by the receiver with correct credentials and private key. CID known to the unauthorized user could not be used to access meaningful content. This proves that the framework is confidential sharing and does not expose file content directly.

C. Local Node/Server Software verification.

The proposed framework was rolled out on all local services:

- IPFS was run on a local node.
- Smart contract worked on a personal block chain system.
- The application backend was locally hosted.

Observed result:

As has been mentioned above, all the workflow of the system (i.e., Encryption > IPFS File Storage > Registering on a Blockchain > Access Verification > File Retrieval and Decrypting) was successfully carried out with only the help of the Local Environment. This proves that the solution is not dependent on a Central Cloud Service; as such, eliminating

the single point of failure with respect to conventional File Sharing Services.

D. Exchange and Authorization Results.

Each file upload was given a new session key of AES. RSA-based key wrapping i.e. AES key encrypted with RSA public key of receiver was used to transmit this AES key securely.

Observed result:

The RSA private key was used to decrypt the AES key by receiver successfully. All the attempts to use a false key as a private key did not work to recover AES key. It therefore means that although IPFS was used to obtain ciphertext, it cannot be decrypted without legitimate private key. This attests to secure exchange of keys and rigid authorization.

E. Dual-Hash Integrity Verification Results.

Dual-hash verification was used to guarantee integrity on two levels:

- H 1 of encrypted file (ciphertext integrity)
- H 2 metadata JSON (context integrity: receiver ID, timestamp, access policy)

Two scenarios of tampering were put to test:

- Alteration of encrypted file (ciphertext)
- Metadata fields modification.

Observed Result:

HFIM missed H1 and download request was rejected because of ciphertext modification. Placing metadata modification led to H2 mismatch and verification. Consequently, dual-hash technique detected file tampering as well as undesired modifications done to metadata that was appended to the files. This has serious implications since this confirms the assumption that the dual-hashing approach has stronger protection than the traditional single-hashing approach.

F. Blockchain Ledger logging and auditability.

Blockchains hold records of irreversible records of:

- CID mapping
- Dual hashes
- Timestamps
- Access policy and identification/token of the receiver.

Overall Result:

The metadata and integrity proofs of files stored in the blockchain cannot be removed after they have been stored on them. Registration of the file and the events of access are both stored in the blockchain and are auditable. Therefore, this creates accountability and traceability within a safe corporate environment of all business transactions.

G. Rule-Based Anomaly Detection Performance.

Rule checks, which were used to detect anomalies in access behaviour in the anomaly detection module, included:

- Attempts at rapid access (repeated).
- Equipment incompatibility / illegal client equipment.
- Abnormal access profile outside threshold.

Observed result:

The successful admission of normal patterns of access was allowed. Suspicious accesses were blocked and alarmed. This enhanced security even in cases whereby attacker tried to use multiple download requests. This affirms that anomaly detection enhances enforcement over and above the existing access control.

VI. Conclusion and Future Scope

The framework for decentralized secure file sharing is presented as a combination of the following elements. Adaptive AES Encryption, RSA Key Wrapping, Dual Hash Integrity Verification, and Blockchain Audit Logging IPFS for decentralized storage of files Rule-Based Anomaly Detection. Each of these technologies is an integral part of the framework to enhance the confidentiality and security of files when being shared from one entity to another. The use of Dual Hash Verification allows the receiver to verify the integrity of both the Encrypted file and its associated metadata; thus, any attempts to tamper with the encrypted files will be detected and the same goes for any attempts to manipulate metadata.

The Hybrid Design consists of the storage of Lightweight References (file references) and Integrity Proofs on the Blockchain, while all other files are stored in IPFS. This design allows for scalability and eliminates the need for a centralized cloud server. The authentication and verification of Prototype Validation were confirmed by setting up a Local Blockchain Network and using IPFS enabled the implementation of Secure File Sharing, the ability to Verify Who Has Access to the File, Audit Logs of Files Shared, and Stop Unauthentic Access in a Decentralized Practical Environment. Thus, the proposed framework for exchanging sensitive information has proven to be an effective and secure method in academia, businesses, and government.

Future Scope:

- Addition of Real Cloud and Edge Environments.

  The prototype under development is tested locally. The framework can be used in future work to implement in real enterprise setups such as cloud-edge deployments of IoT data sharing, hospital data exchange, and government secure portal.
- Support for Data Compliance and Audit Reports:

  This Work Item is intended to support the building of an automated system capable of producing audit reports for various compliance standards, such as those defined by ISO, GDPR, etc. Records of compliance will be verifiable via the use of Blockchain audit trails.
- Parallel Processing and Optimization of performance.

  In the case of big files, the time of encryption and upload can be extended. Parallel encryption, file chunking and pipelined upload can also be incorporated in future work to accelerate the

performance. Blacklisting of coded metadata can decrease blockchain storage as well.

- Group and Multi-User File sharing.

  The current workflow is concerned with sender to receiver sharing. The system can be extended in the future to group sharing that is secure with the ability to add and remove group members (dynamic group membership: join/leave), group key distribution, and revocation without re-encryption of all the files.

VII. References

[1] W. Jansen and T. Grance, "Guidelines on Security and Privacy in Public Cloud Computing," *NIST Special Publication 800-144*, Dec. 2011.

[2] K. S. Sahoo, D. Puthal, M. Tiwary, J. J. P. C. Rodrigues, and B. Sahoo, "An early detection of low rate DDoS attack to SDN based data center networks using information distance metrics," *Future Generation Computer Systems*, vol. 89, pp. 685–697, Dec. 2018, doi: 10.1016/j.future.2018.07.017.

[3] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[4] H. Guo, W. Li, M. Nejad, and C. Shen, "Multi-Authority Attribute-Based Access Control with Smart Contract," arXiv:1903.07009, 2019.

[5] J. Benet, "IPFS—Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[6] M. Naz, M. Al-Zahrani, R. Khalid, N. Javaid, and M. Guizani, "A Secure Data Sharing Platform Using Blockchain and Interplanetary File System," *Sustainability*, vol. 11, no. 24, Art. no. 7054, 2019, doi: 10.3390/su11247054.

[7] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," *FIPS PUB 197*, Nov. 2001.

[8] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," *FIPS PUB 180-4*, Aug. 2015.

[9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[10] R. Afzal and R. K. Murugesan, "Rule-Based Anomaly Detection Model with Stateful Correlation Enhancing Mobile Network Security," *Intelligent Automation & Soft Computing*, vol. 31, no. 3, pp. 1825–1841, 2022, doi: 10.32604/iasc.2022.020598.

[11] H. Huang, H. Chang, and S. Chen, "A Secure File Sharing System Based on IPFS and Blockchain," in *Proc. ACM Int. Conf. on Blockchain and Secure Critical Infrastructure (BSCI)*, 2020, pp. 1–6, doi: 10.1145/3409934.3409948.

[12] G. Zyskind, O. Nathan, and A. Pentland, "Decentralizing Privacy: Using Blockchain to Protect Personal Data," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2015, pp. 180–184, doi: 10.1109/SPW.2015.27.

[13] N. Szabo, "Smart Contracts: Building Blocks for Digital Markets," 1996. (Online manuscript).